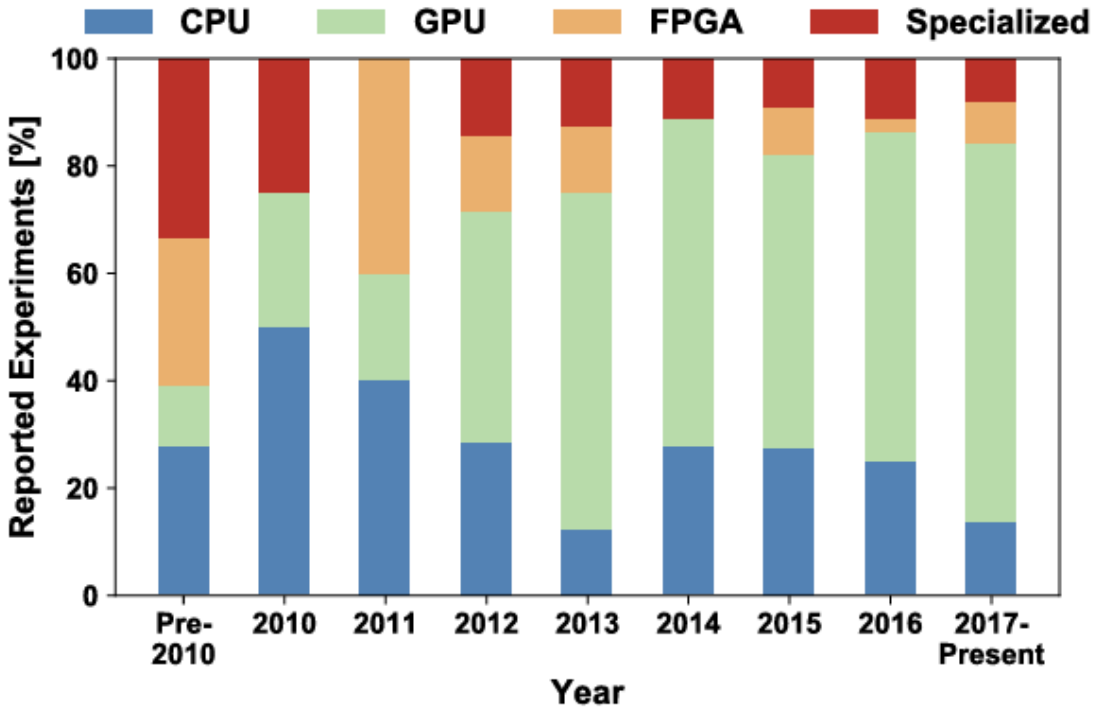
Parallel Deep Learning

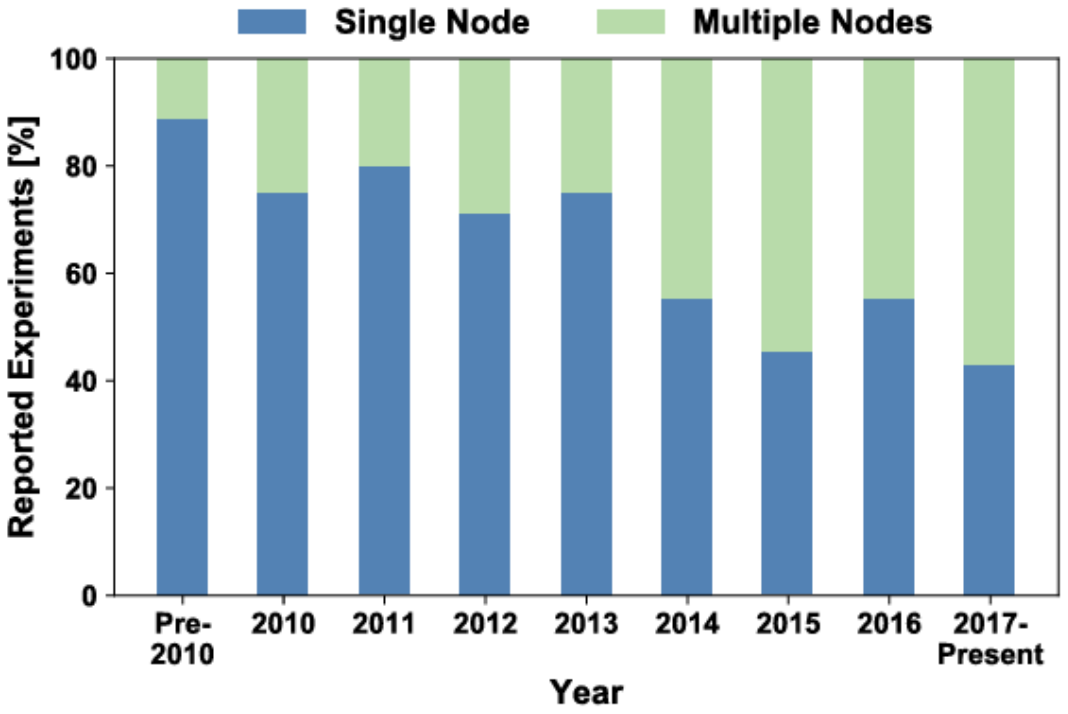
Lecture 27

April 22, 2026

Training on Multiple Nodes



(a) Hardware Architectures



(b) Training with Single vs. Multiple Nodes

Demystifying Parallel and Distributed Deep Learning: An In-depth Concurrency Analysis, Ben-Nun et al., ACM CSUR 2019

Going deeper with convolutions, Szegedy et al., 2014

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

Table 1: GoogLeNet incarnation of the Inception architecture

for enhancing the performance of CNN. We design a streamlined version of GoogLeNet [13], which was original proposed for image classification in recent years with very deep architecture, for HCCR (denoted as HCCR-GoogLeNet). The HCCR-GoogLeNet we used is 19 layers deep but involves with only 7.26 million parameters. Experiments were conducted using the

After the final convolutional layer, there are two huge fully connected layers with 4096 outputs. These layers require nearly 1GB model parameters. Because of the limited memory in early GPUs, the original AlexNet used a dual data stream design, so that each of their two GPUs could be responsible for storing and computing only its half of the model. Fortunately, GPU memory is comparatively abundant now, so we rarely need to break up models across GPUs these days (our version of the AlexNet model deviates from the original paper in this aspect).

High Performance Offline Handwritten Chinese Character Recognition Using GoogLeNet and Directional Feature Maps

https://d2l.ai/chapter_convolutional-modern/alexnet.html

Dataset

	Competition	Size
1	Diabetic Retinopathy Detection	82.2 GiB
2	American Epilepsy Society Seizure Prediction Challenge	59.6 GiB
3	Avito Duplicate Ads Detection	48.4 GiB
4	Microsoft Malware Classification Challenge (BIG 2015)	35.3 GiB
5	GE Flight Quest	34.4 GiB
6	Draper Satellite Image Chronology	32.9 GiB
7	CHALEARN Gesture Challenge	31.1 GiB
8	Second Annual Data Science Bowl	29.7 GiB
9	Flight Quest 2: Flight Optimization, Main Phase	25.8 GiB
10	Belkin Energy Disaggregation Competition	20.0 GiB

Source: Kaggle

OOM

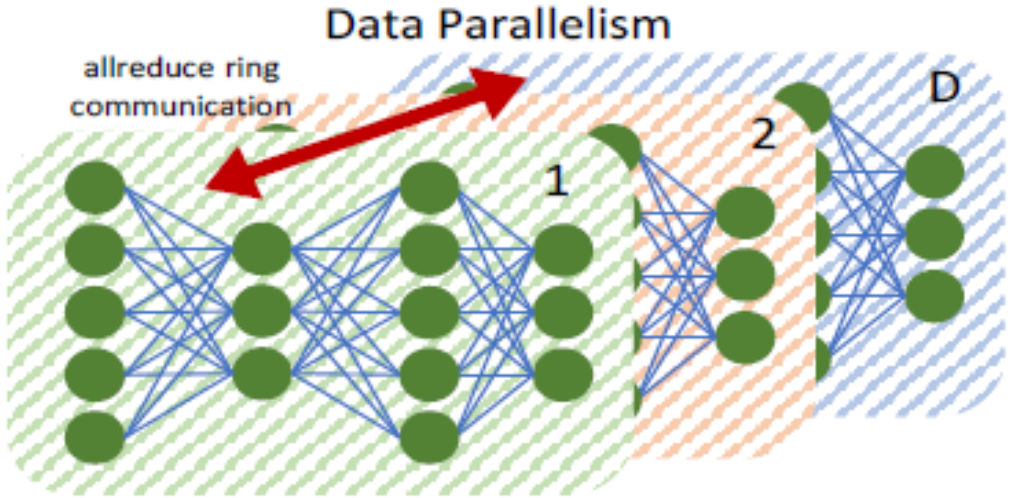
As a rough but not guaranteed estimate, you should have at least `4*num_parameters` bytes of GPU memory in order to run in `--fp16` mode and at least `8*num_parameters` bytes of GPU memory in order to run in fp32 precision. You should also have at least `12*num_parameters` bytes of CPU memory for model loading and engine building and serialization. For example, for a 6B model, you should have >=24GB GPU memory for `--fp16`, or >=32GB GPU memory for fp32, and >=72GB CPU memory. It is recommended to run `--fp16 --use-cache` to optimize engine build and inference.

Source:
https://github.com/NVIDIA/TensorRT/tree/release/9.1/demo/HuggingFace

Distributed Deep Learning

- Data Parallelism
- …

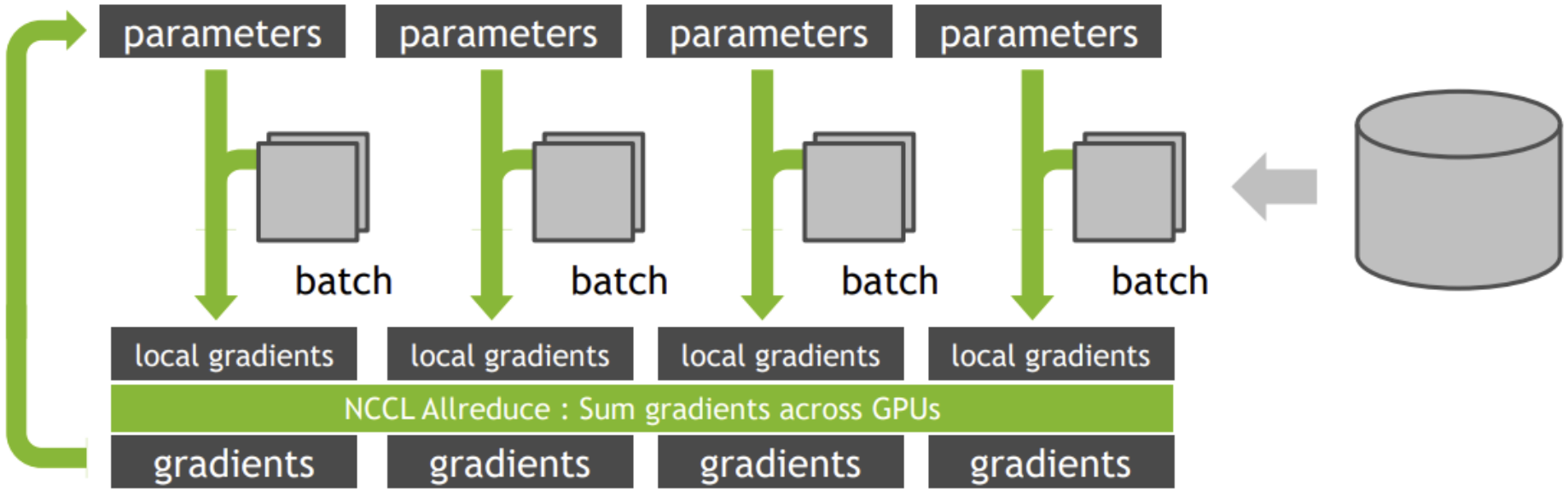
- Partitions the dataset
- Duplicates the DL model
- Training on local data
- Update weights



HammingMesh: A Network Topology for Large-Scale Deep Learning, Hoefler et al., SC22

Distributed DL Frameworks

- Horovod
- Tensorflow + Horovod
- PyTorch + Horovod
- …



https://www.olcf.ornl.gov/wp-content/uploads/2019/12/Summit-NCCL.pdf

Introduction

Accelerating distributed deep neural network training with pipelined MPI_Allreduce, Castello et al., Cluster Computing, 2021

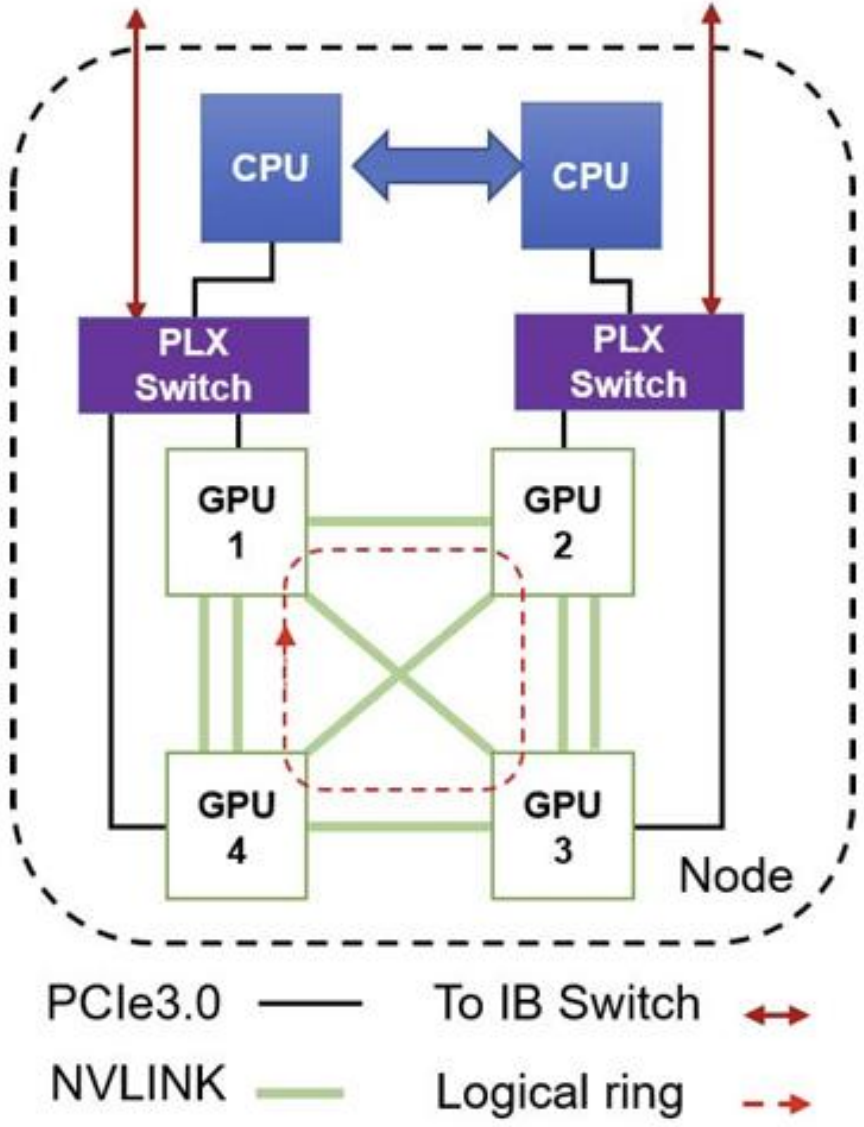
MPI (Message Passing Interface) [28] is the *de facto* standard for distributed high performance computing (HPC) applications. Therefore, it has been naturally adopted as the communication layer for distributed training frameworks such as Google's TensorFlow (TF) [1], TF+Horovod (HVD) [25], and PyTorch [24]. The MPI application programming interface (API) comprises a large variety of peer-to-peer and collective communication primitives. Among these, the DP scheme for distributed DNN training basically relies on the blocking `MPI_Allreduce` primitive, which internally reduces a collection of local values broadcasting the global result to all processes participating in the communication.

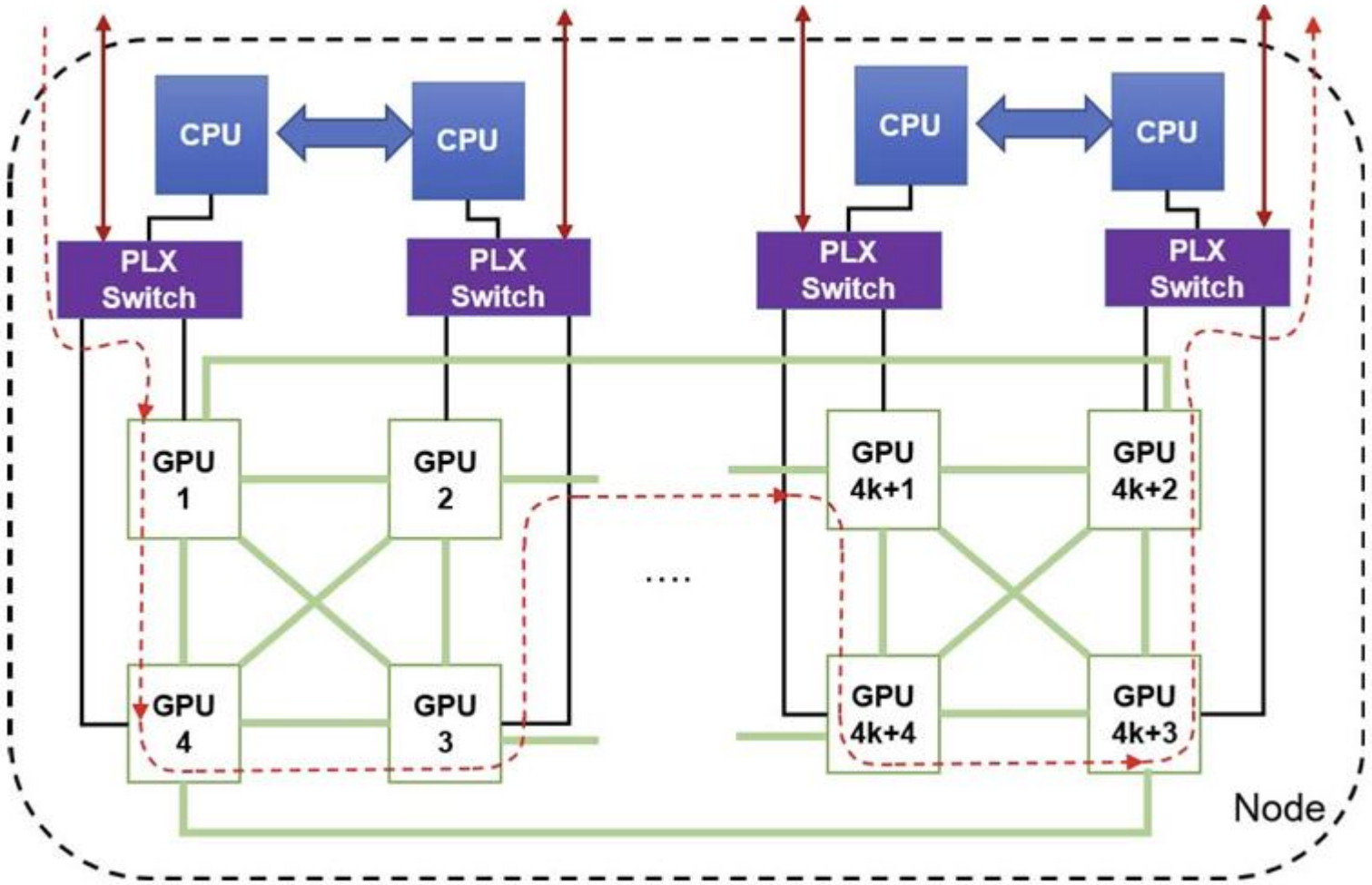
Quite a few papers address this limitation in the context of ML/DL

- Bandwidth Optimal Allreduce Algorithms for Clusters of Workstations, Patarasuk et al., JPDC 2009
- Hierarchical Distributed-Memory Multi-Leader MPI_Allreduce for Deep Learning Workloads, Nguyen et al., ISCNW 2018
- Efficient MPI_AllReduce for large-scale deep learning on GPU-clusters, Nguyen et al., CCPE 2019
- ….

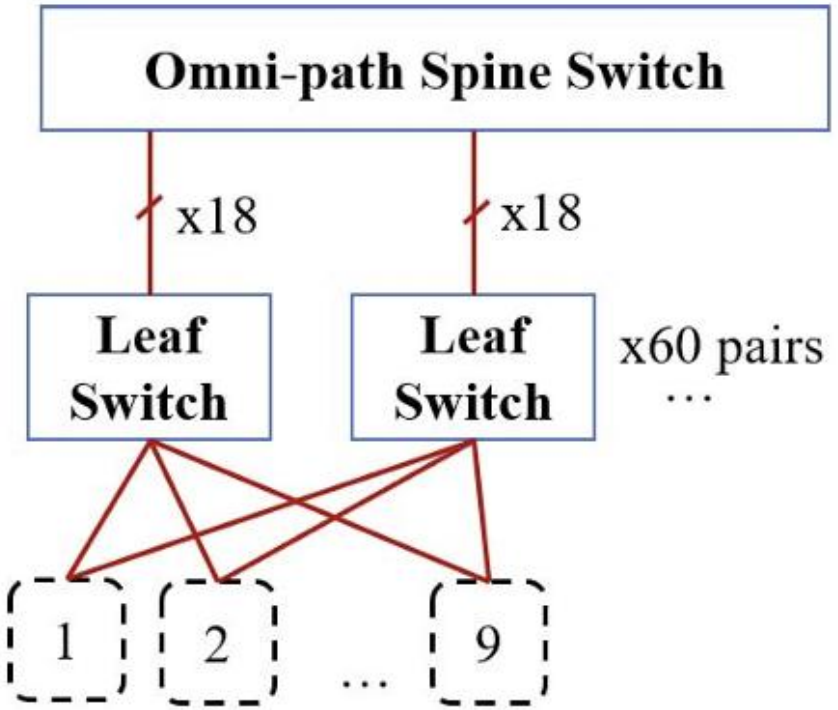
Multi-GPU Setup

Network	Latency α (second per hop)	β = Bandwidth^{-1} (second per byte)
Mellanox EDR IB [18]	90×10^{-9}	8×10^{-11}
PLX swich + PCIe link [19]	110×10^{-9}	6.25×10^{-11}
NVLINK	≈ 0	2×10^{-11}

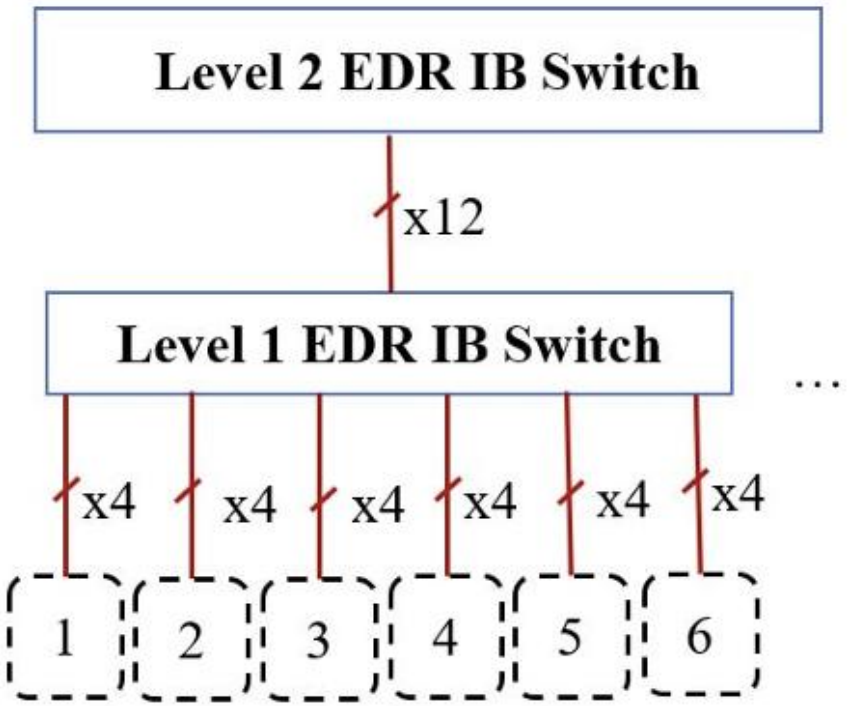




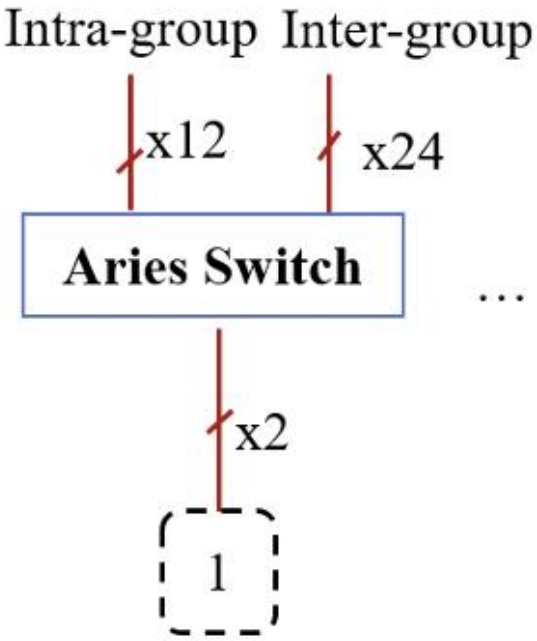
Network



TSUBAME3



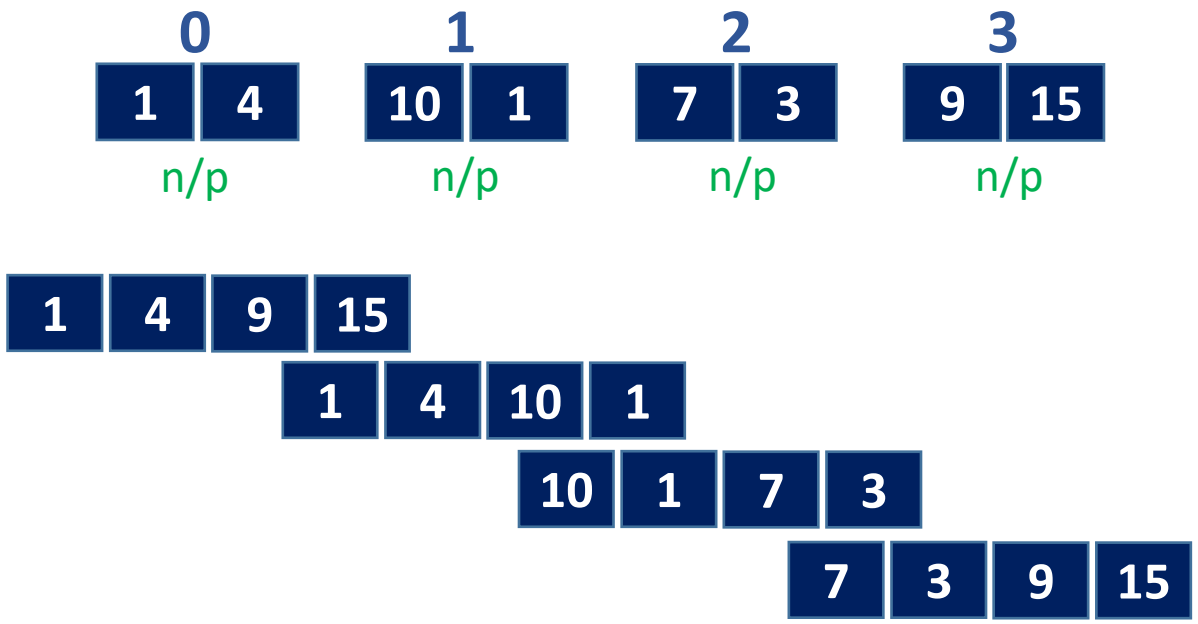
NVCLUSTER



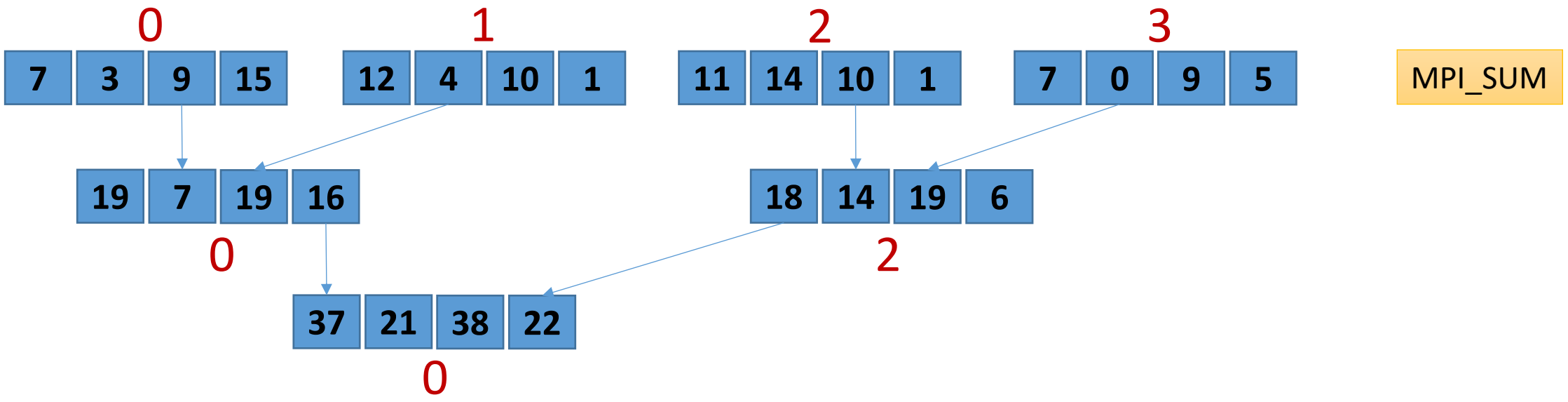
DRAGONFLY
(4 switches/group)

Allgather – Ring Algorithm Recap

- Every process sends to and receives from everyone else
- Assume p processes and total n bytes
- Every process sends and receives n/p bytes
- Time
  - (p – 1) * (L + n/p*(1/B))

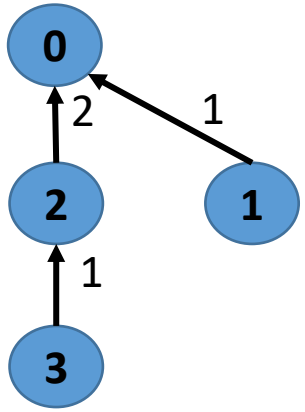


Reduce Algorithm – Recursive doubling Recap

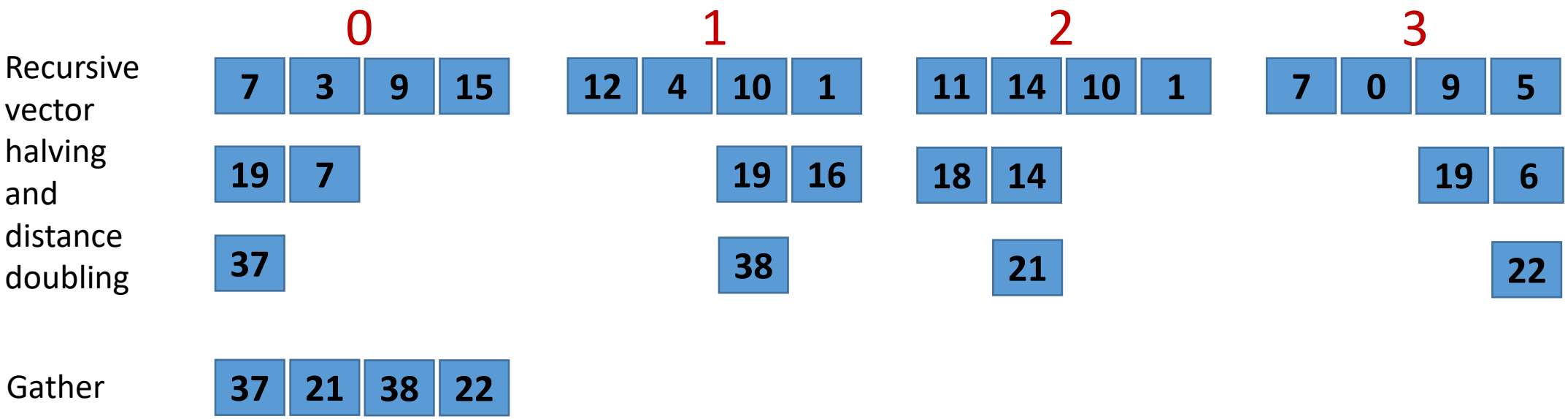


Time: log p (L + n*(1/B) + n*c)
L = latency, B = bandwidth
c = compute cost per byte

Used for short messages



Reduce – Rabenseifner's Algorithm Recap



Time:

log p * L + (p-1)/p*(n/B) + (p-1)/p*n*c (reduce-scatter) +

log p * L + (p-1)/p*(n/B) (gather using binomial)

n = data size

p = #processes

c = compute cost per byte

L = latency

B = bandwidth

MPI_Gather (last step in previous slide)

node 0: recv B => AB

node 1: recv F => EF

node 2: recv D => CD

node 3: recv H => GH

node 4: send B

node 5: send F

node 6: send D

node 7: send H

node 0: recv CD => ABCD

node 1: recv GH => EFGH

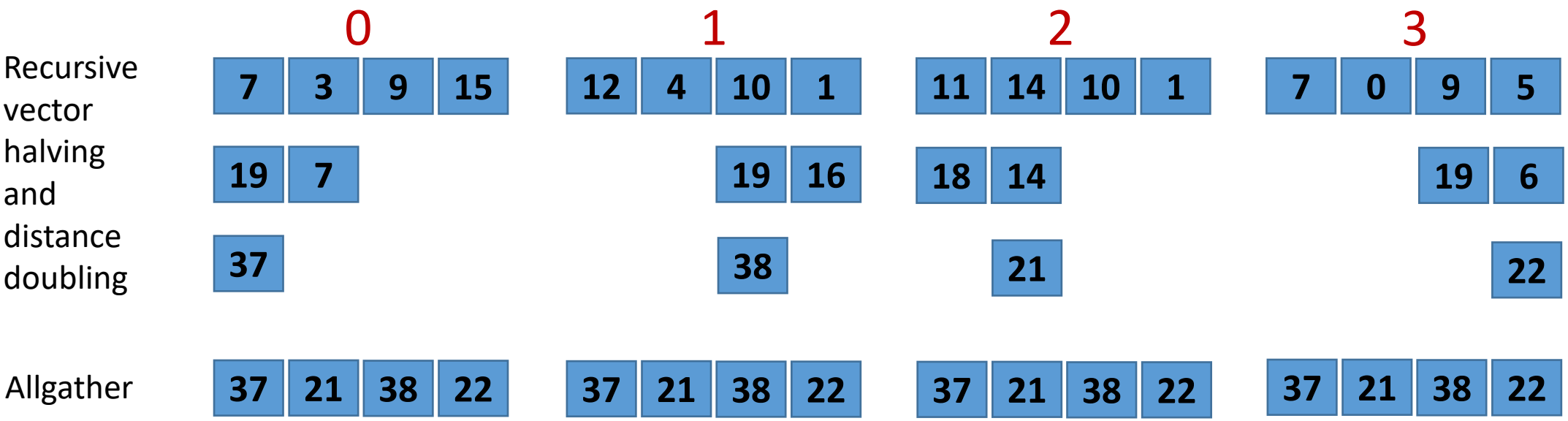
node 2: send CD

node 3: send GH

node 0: recv EFGH => ABCDEFGH

Credit: cds.iisc.ac.in

Allreduce – Rabenseifner's Algorithm Recap



Time:

log p * L + (p-1)/p*(n/B) + (p-1)/p*n*c (reduce-scatter) +

log p * L + (p-1)/p*(n/B) (allgather using recursive vector doubling and distance halving)

n = data size L = latency

p = #processes B = bandwidth

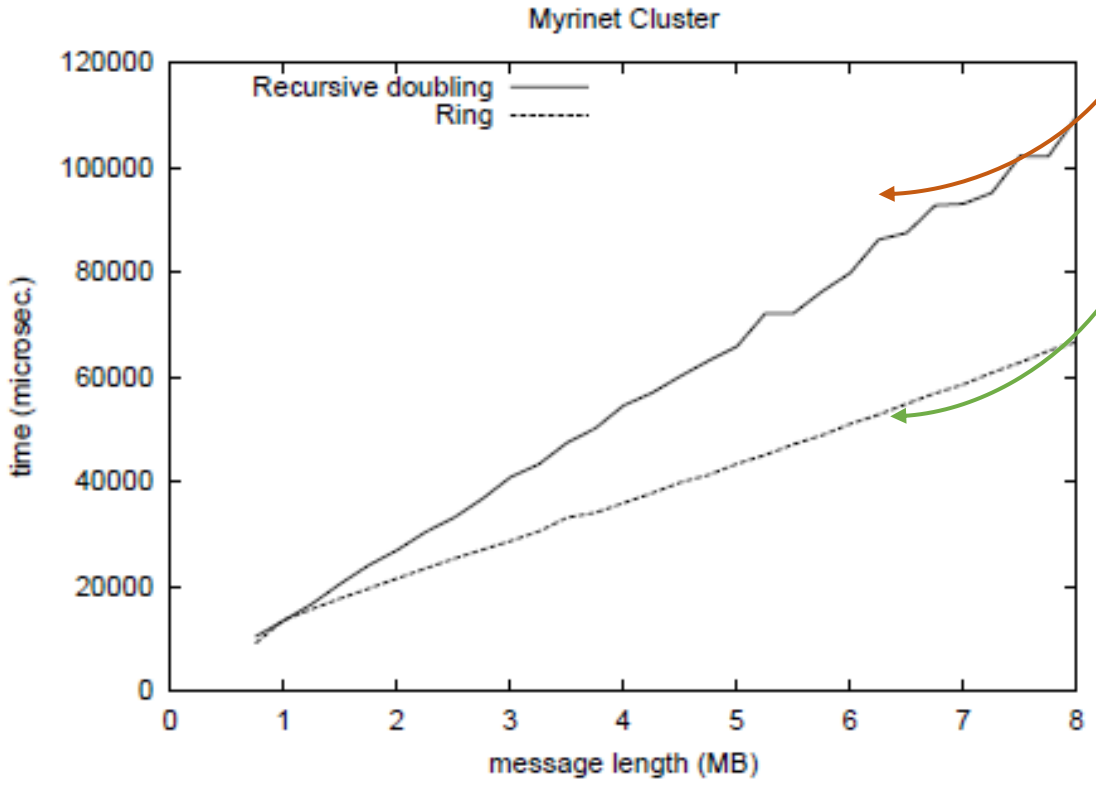
c = compute cost per byte

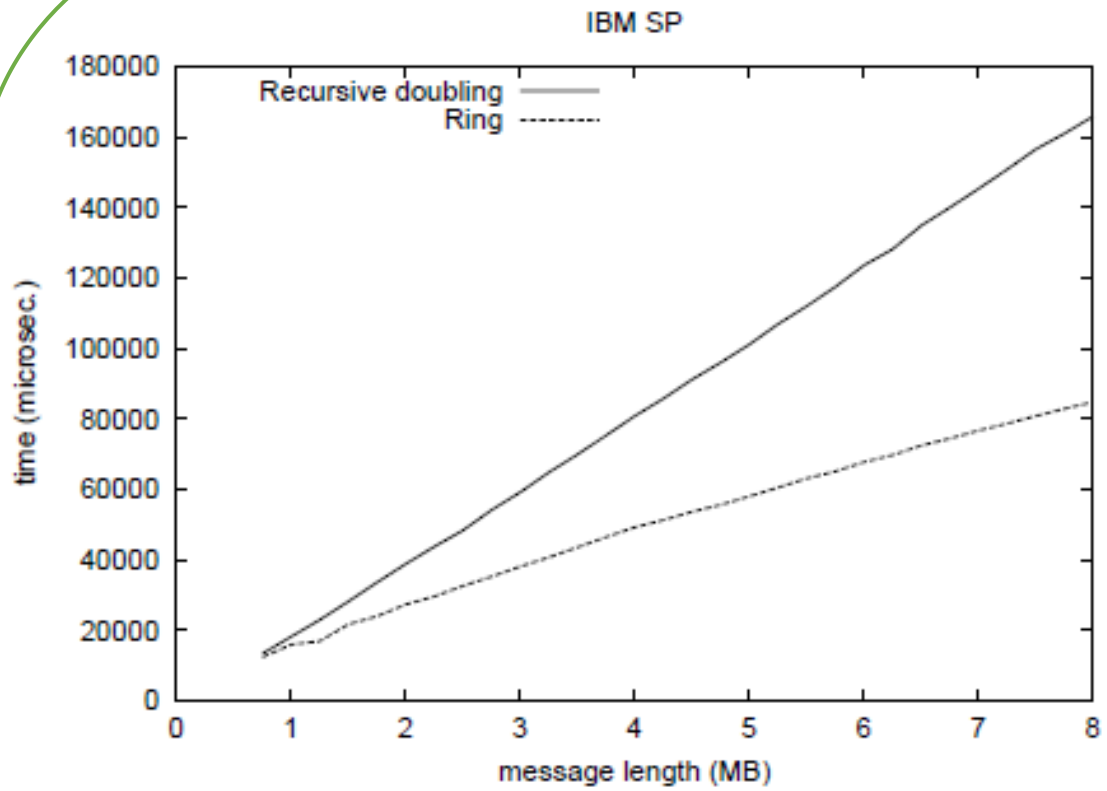
Performance Comparison on 64 nodes Recap

Why does ring perform better?

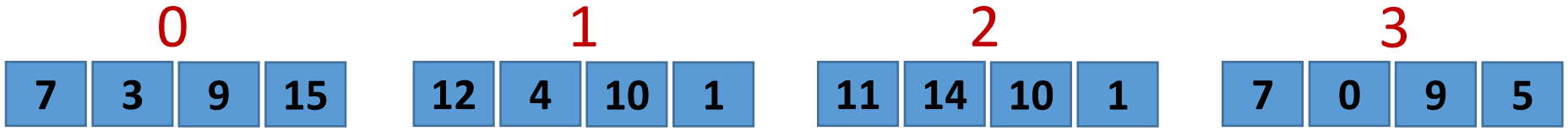
(log p) * L + (p-1)*n/p*(1/B)

(p-1) * L + (p-1)*n/p*(1/B)





MPI_Reduce/MPI_Allreduce (Ring)



i^{th} segment is reduced by i^{th} process following a ring algorithm (i.e. rank r sends to rank (r+1) mod P)

Gather/
Allgather

$$2(p-1)\alpha + 2\,p-1/p\,*N/\beta + p-1/p\,*N\gamma$$

Efficient MPI_AllReduce for large-scale deep learning on GPU-clusters, Nyugen et al., CCPE, 2019

Illustration of MPI_Allreduce (Ring)

a0	b0	c0	d0
a1	b1	c1	d1
a2	b2	c2	d2
a3	b3	c3	d3

a0	b0	c0+c3	d0
a1	b1	c1	d0+d1
a1+a2	b2	c2	d2
a3	b2+b3	c3	d3

Step 1

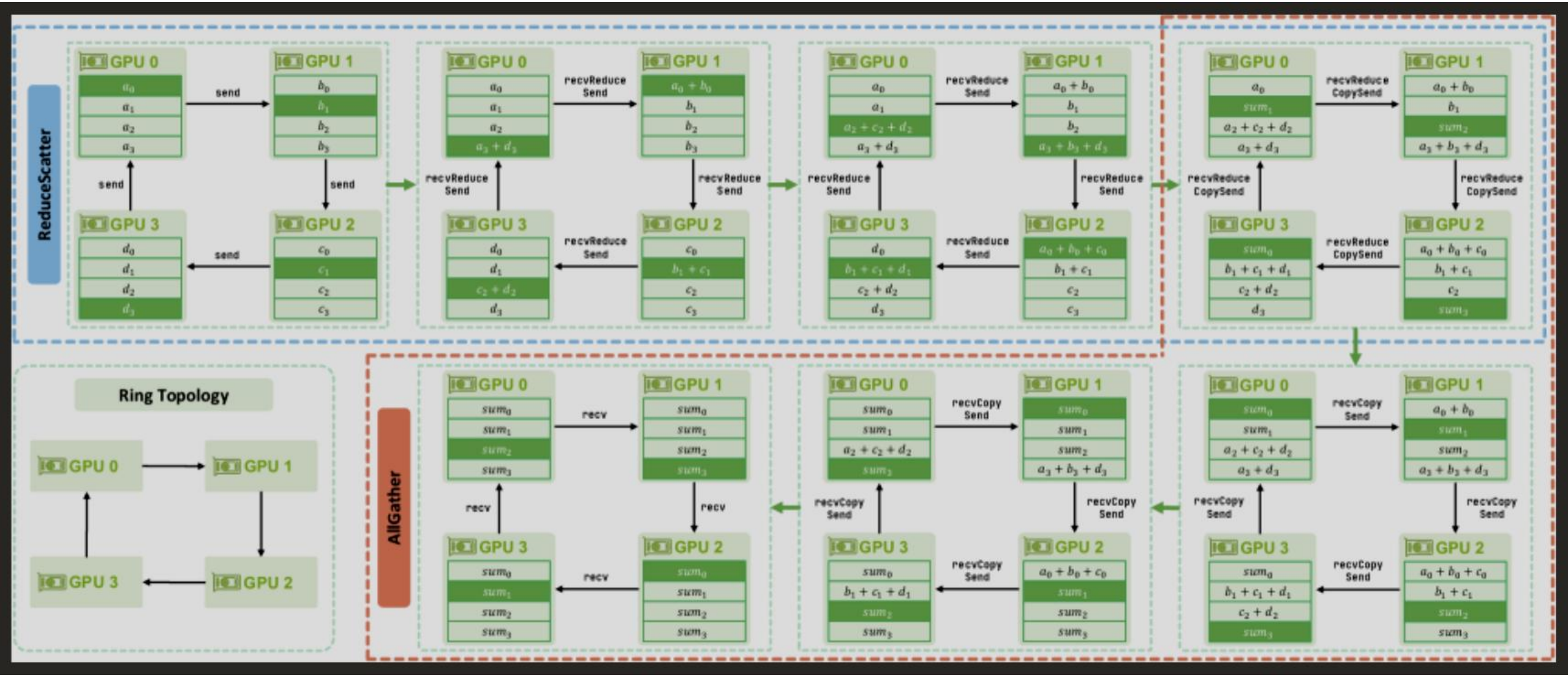
a0	b0+b2+b3	c0+c3	d0
a1	b1	c0+c1+c3	d0+d1
a1+a2	b2	c2	d0+d1+d2
a1+a2+a3	b2+b3	c3	d3

Step 2

Illustration (contd.) of MPI_Allreduce (Ring)

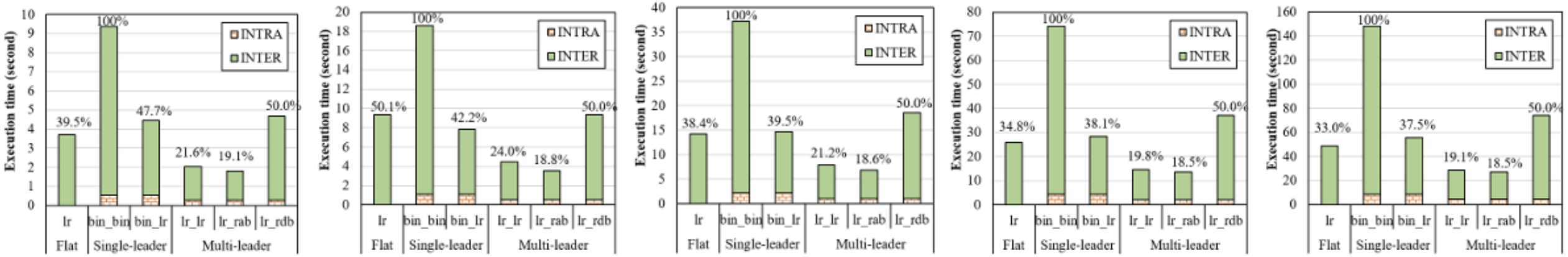
a0+a1+a2+a3	b0+b2+b3	c0+c3	d0
a1	b0+b1+b2+b3	c0+c1+c3	d0+d1
a1+a2	b2	c0+c1+c2+c3	d0+d1+d2
a1+a2+a3	b2+b3	c3	d0+d1+d2+d3

Step 3



[Demystifying NCCL: An In-depth Analysis of GPU Communication Protocols and Algorithms]

Results



(a) 8 MB. (b) 16 MB. (c) 32 MB. (d) 64 MB. (e) 128 MB.

Fig. 3. Execution time versus message size simulated on the 256-GPU system identical to TSUBAME3 architecture.

Algorithms for MPI_Allreduce

Algorithm	Latency factor	Bandwidth factor	Computation factor
Binomial tree	$\log(p)\alpha$	$\log(p)N\beta$	$\log(p)N\gamma$
Recursive-doubling	$\log(p)\alpha$	$\log(p)N\beta$	$\log(p)N\gamma$
Rabenseifner[32]	$2\log(p)\alpha$	$2\frac{p-1}{p}N\beta$	$\frac{p-1}{p}N\gamma$
Logical ring[33]	$2(p-1)\alpha$	$2\frac{p-1}{p}N\beta$	$\frac{p-1}{p}N\gamma$

Pipelined MPI_Allreduce

```
int MPI_Allreduce (const void *sendbuf, void *recvbuf,
int count, MPI_Datatype datatype,
MPI_Op op, MPI_Comm comm)
```

```
int MPI_Iallreduce (const void *sendbuf, void *recvbuf,
int count, MPI_Datatype datatype,
MPI_Op op, MPI_Comm comm, MPI_Request *request)
```

Segmented Non-blocking Allreduce

int MPI_Iallreduce (const void *sendbuf, void *recvbuf,
int count, MPI_Datatype datatype,
MPI_Op op, MPI_Comm comm, MPI_Request *request)

```
for (….)
        MPI_Iallreduce (…count/x…)
```

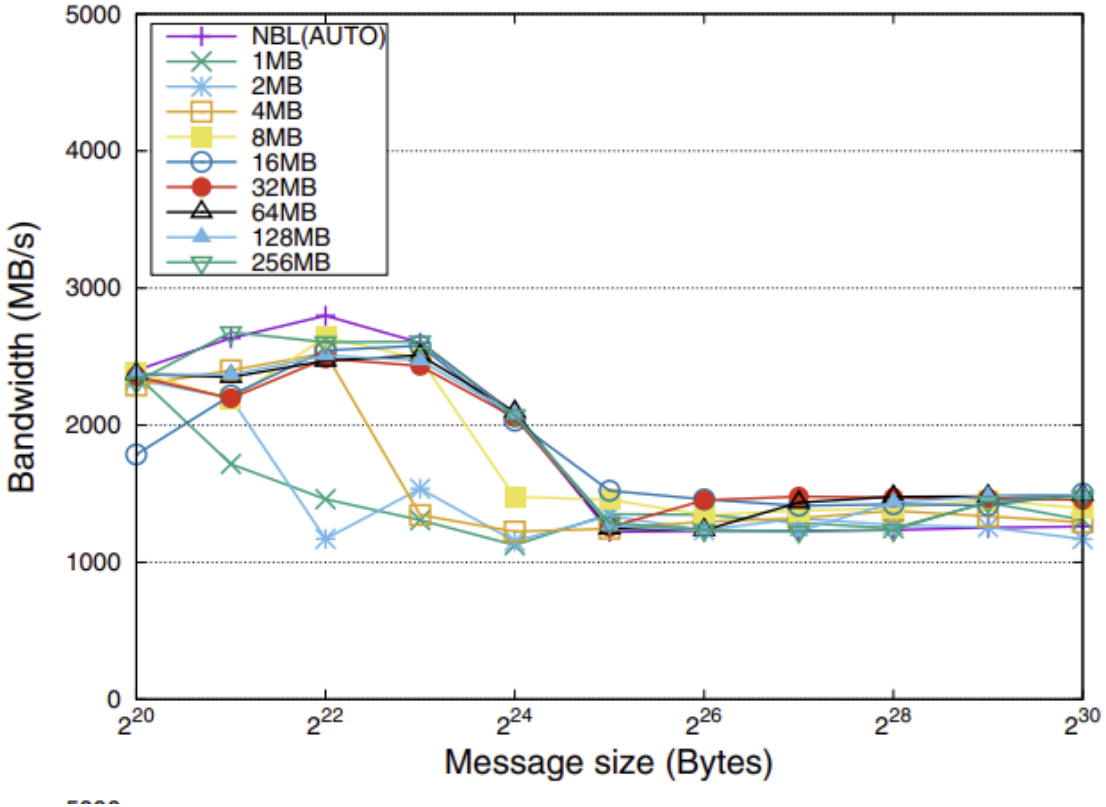
How do we segment?

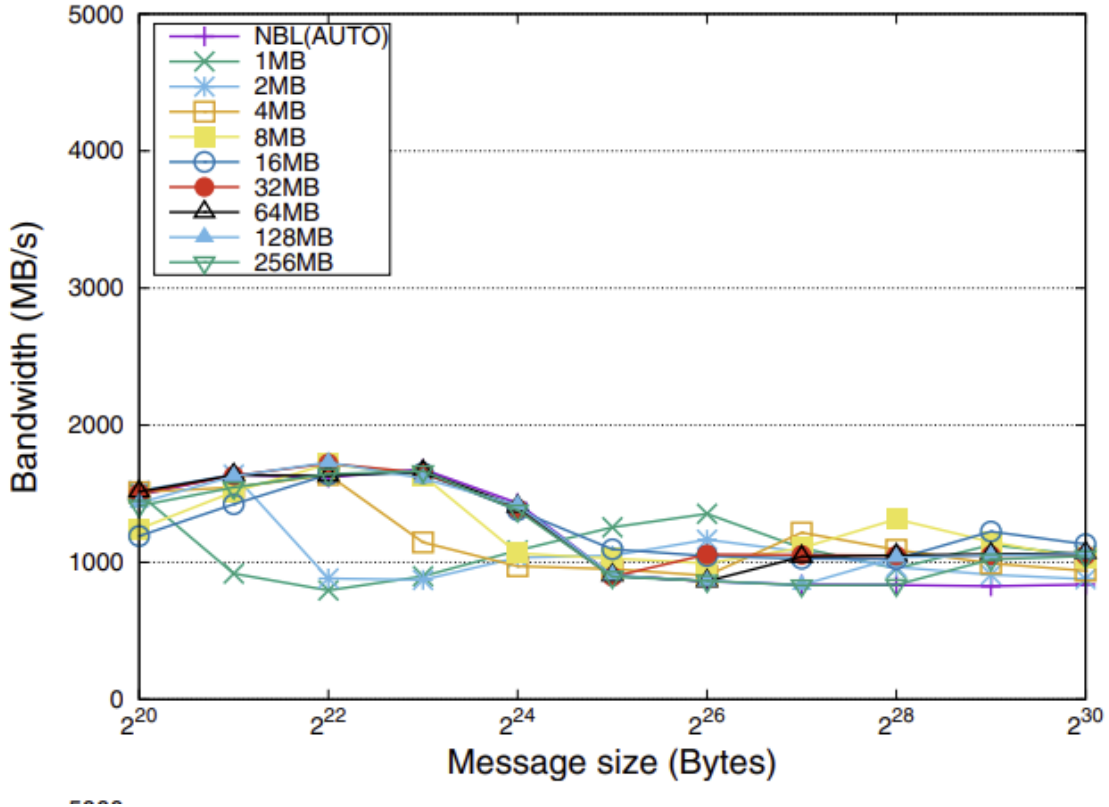
- Segment of fixed size
- Fixed number of segments

Accelerating distributed deep neural network training with pipelined MPI_Allreduce, Castello et al., Cluster Computing, 2021

Performance with Fixed Segment Size (AUTO)

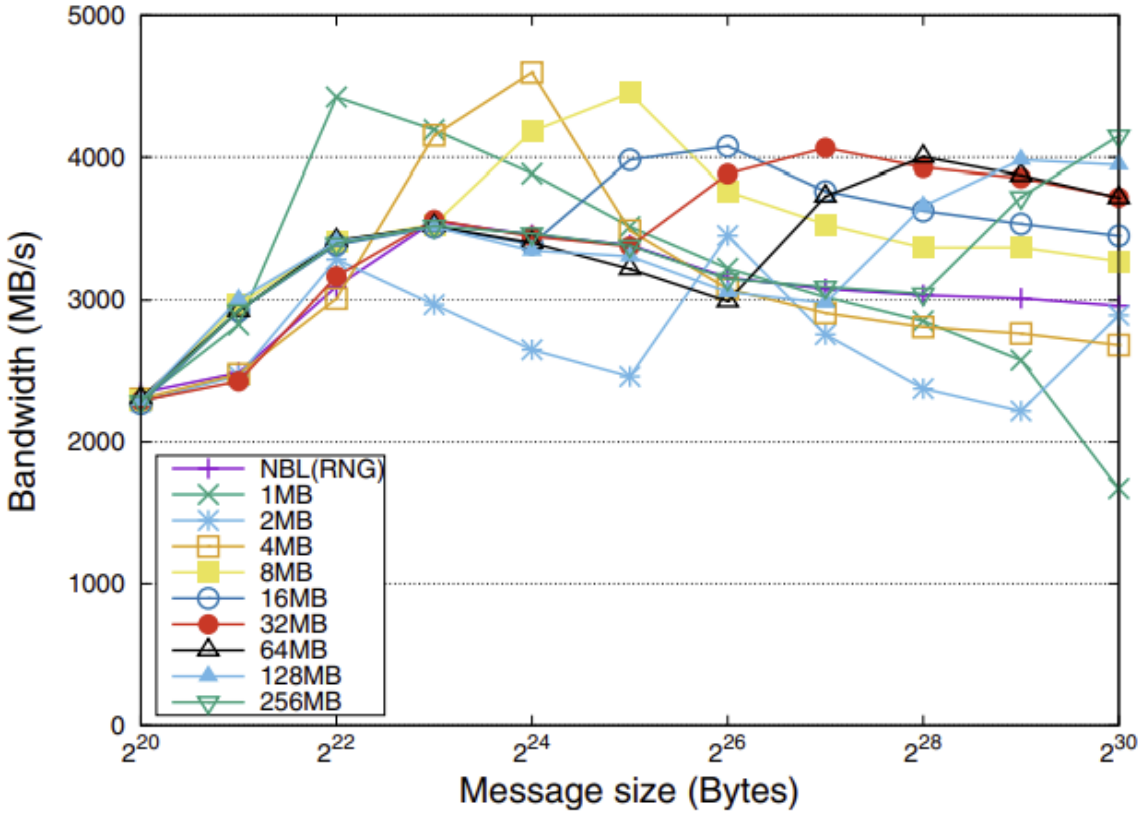
8 and 9 processes

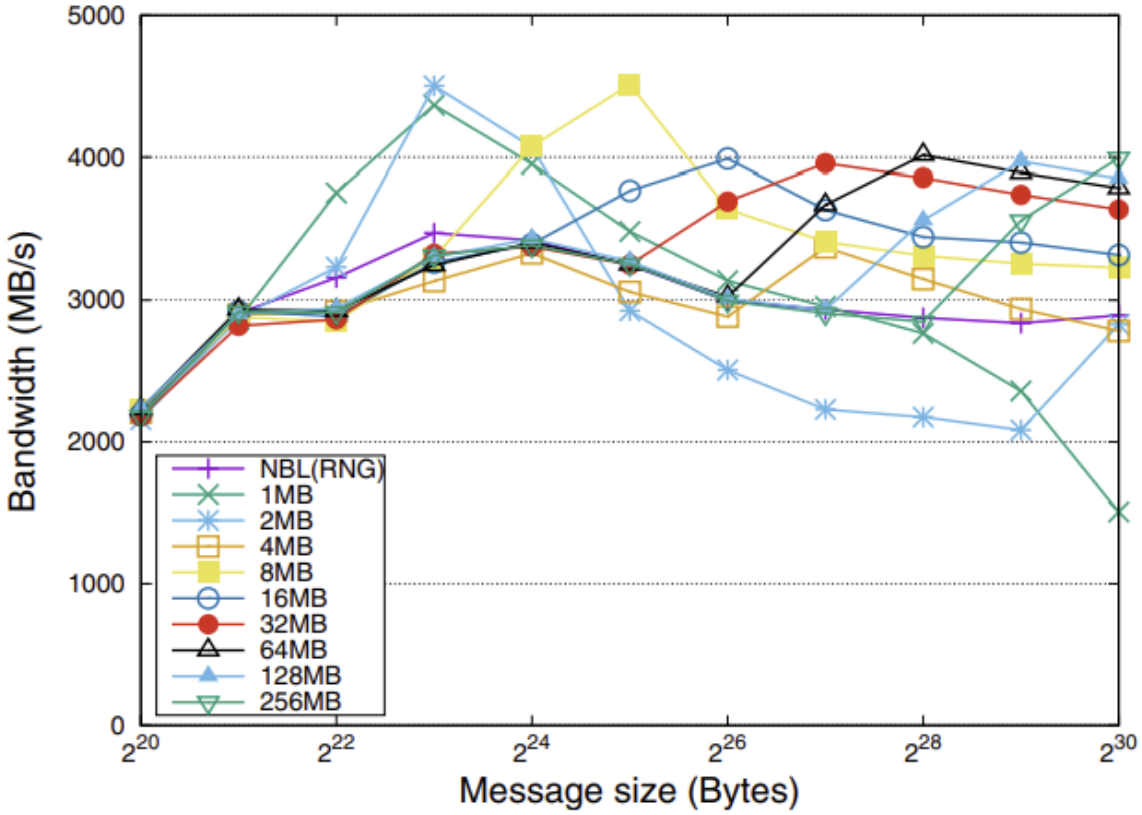




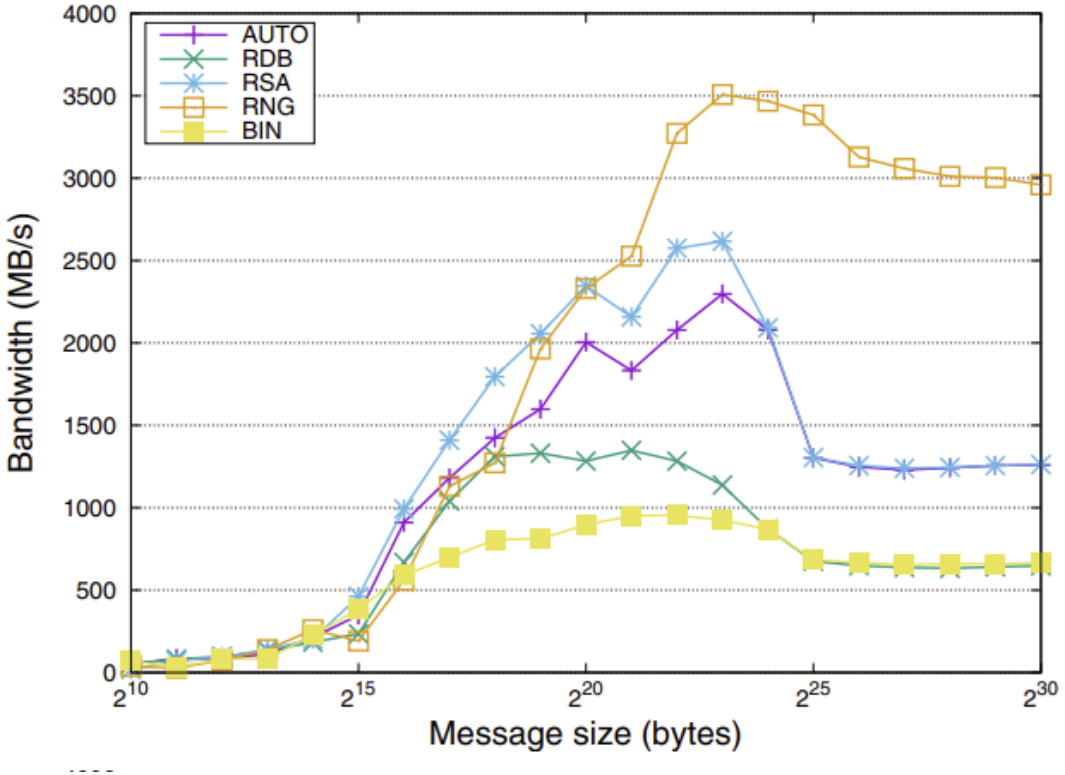
Performance with Fixed Segment Size (RING)

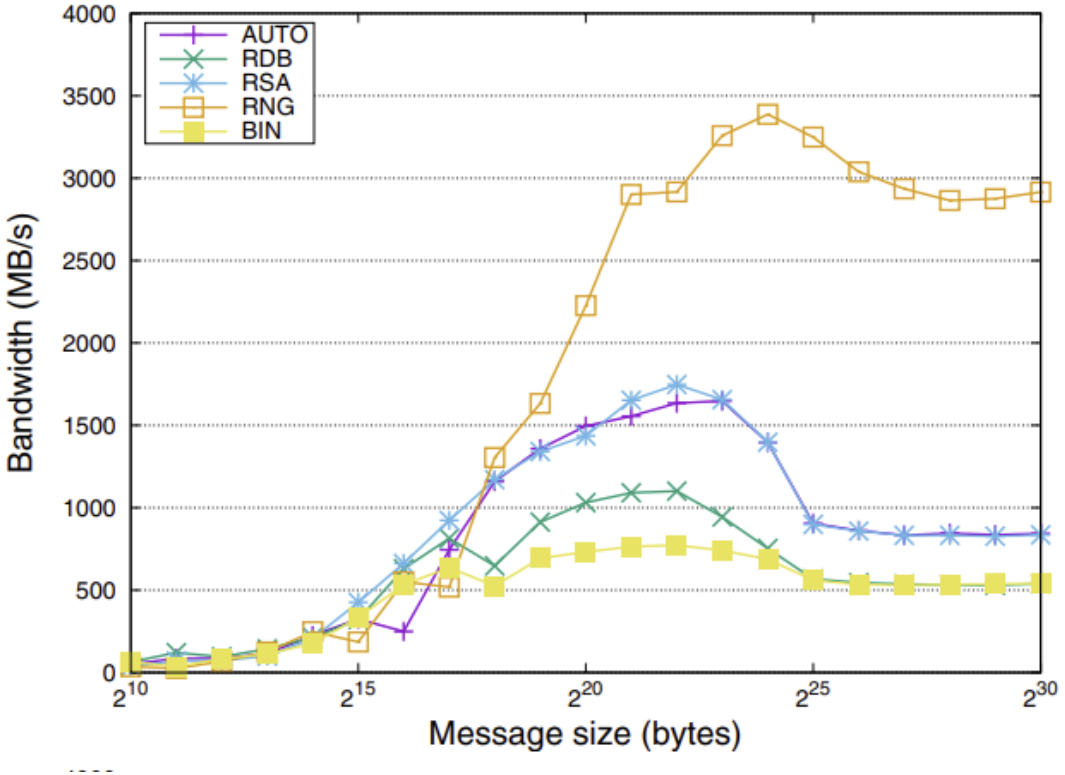
8 and 9 processes





Performance of MPI_Allreduce using OpenMPI implementation (8 and 9 processes)





Performance of TF+HVD (Batch size = 128)

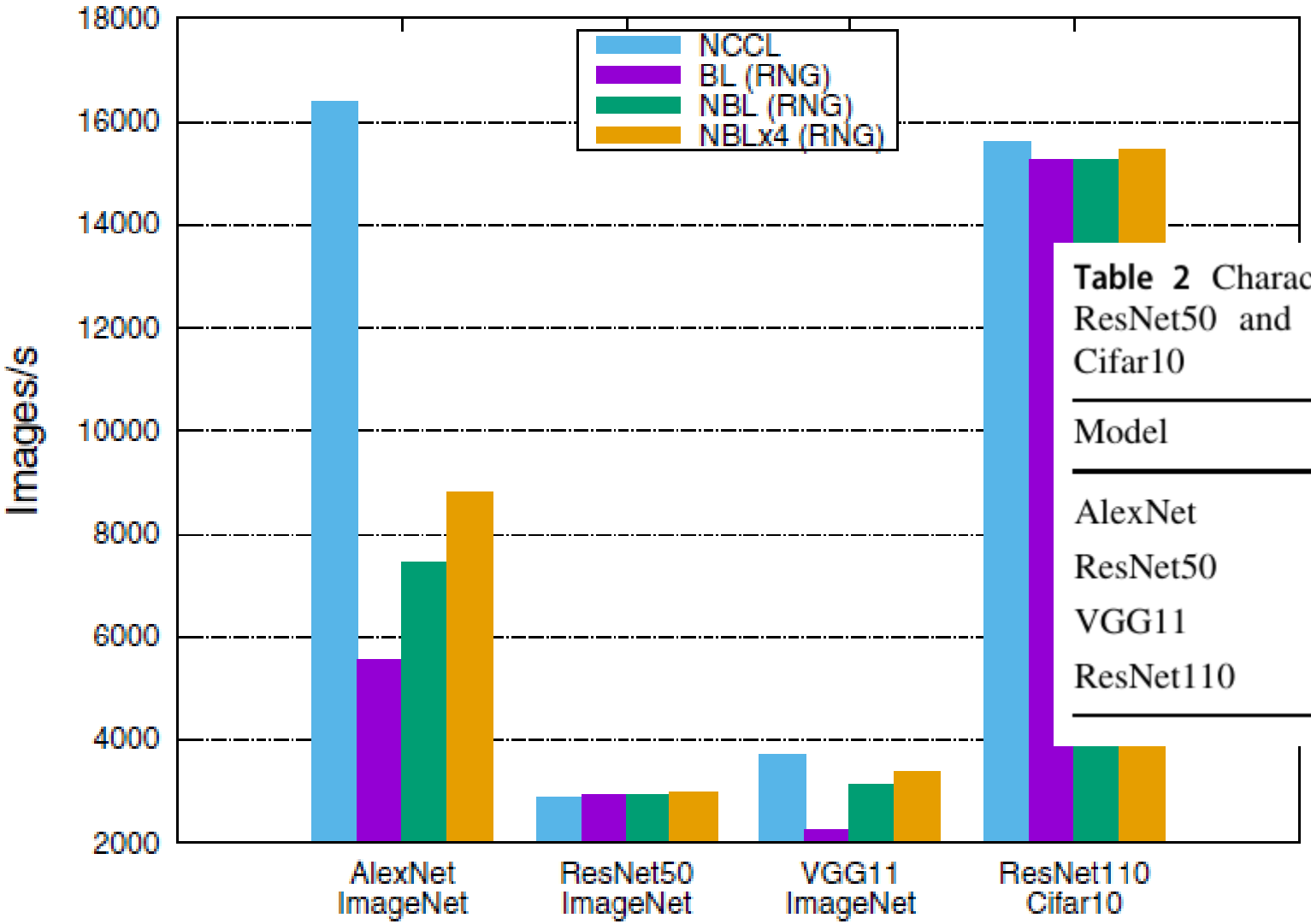
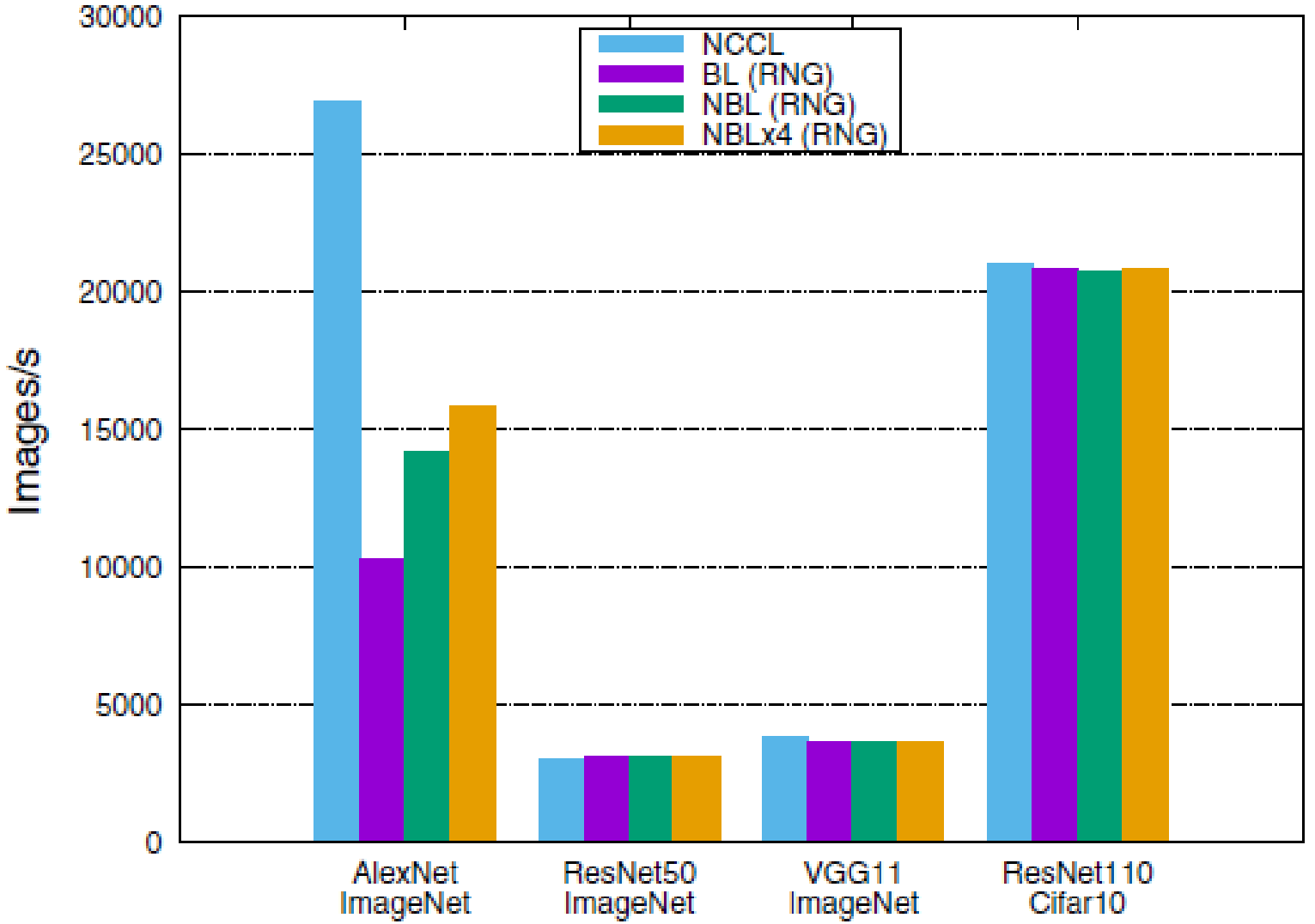


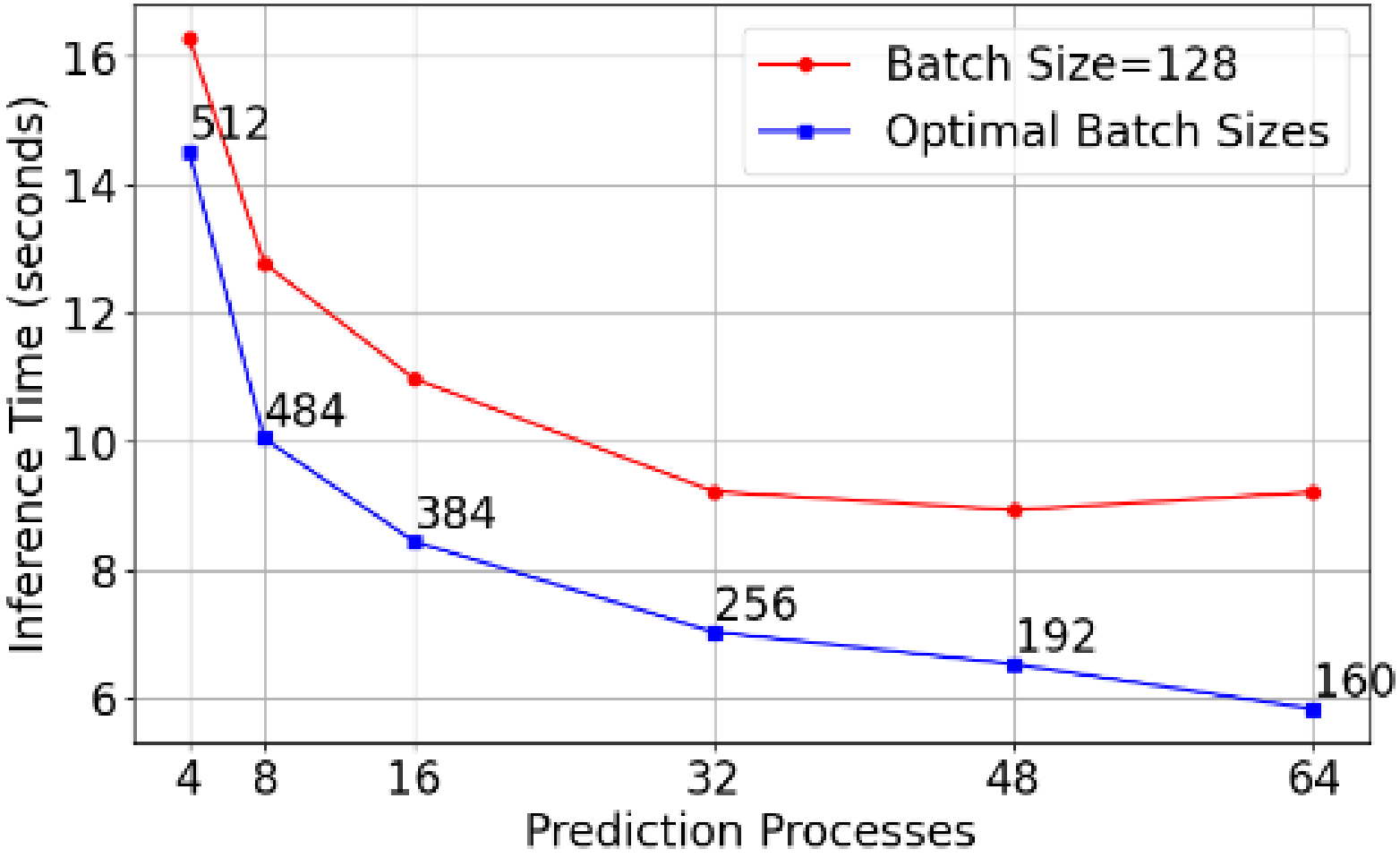
Table 2 Characterization of the DNN models+datasets. AlexNet, ResNet50 and VGG11 employ ImageNet and ResNet110 is for Cifar10

Model	# of layers	Params.	flops $\times b$
AlexNet	22	62,378,344	2,270,512,192
ResNet50	176	25,636,712	8,178,368,512
VGG11	30	132,863,336	15,218,180,096
ResNet110	393	1,747,898	506,832,128

Performance of TF+HVD (Batch size = 256)



Optimal Batch Sizes Vary

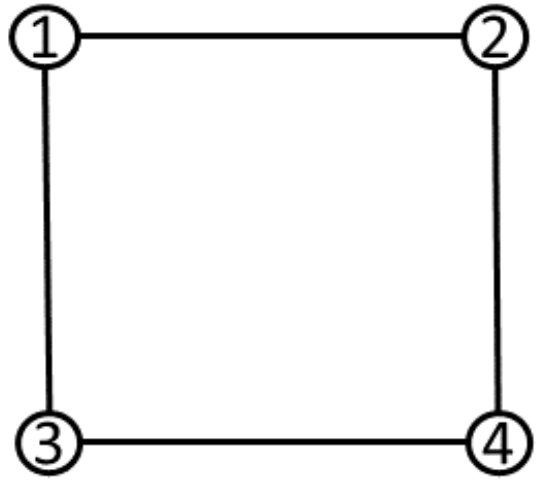


Credit: M.A. Wani

H.W. Q1

MPI_Bcast of 10 KB data (root=0) on the 2D mesh. There are 8 processes placed on the 4 nodes. Ranks 0 and 1 are placed on node 1, ranks 2 and 3 are placed on node 2 and so on. Bandwidth of every link is 1 Gbps. Assume hop=0 between processes in a node. Assume XY routing policy (i.e. messages first traverse in x-dimension, followed by y-dimension). Total time = 4 ms.

Analyze and discuss the maximum #hops, average #hops, hop-bytes and link contention.



H.W. Q2

Compare and contrast recursive doubling algorithm for MPI_Reduce on 8 processes for the following node allocations:

(a) Ranks 0 – 3 are on csews1, ranks 4 – 7 are on csews2

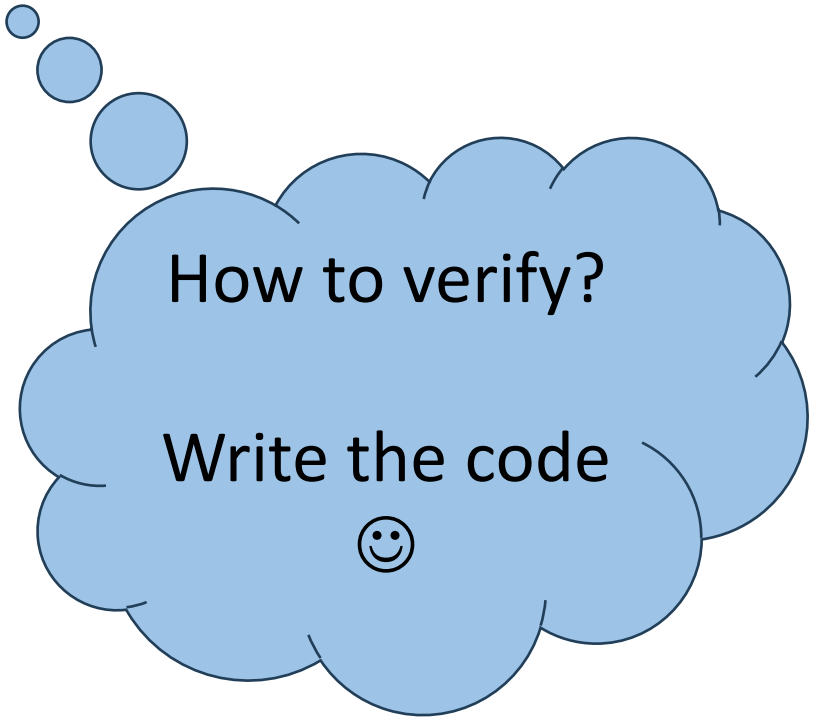
(b) Even ranks are on csews1, odd ranks are on csews2

H.W. Q3: 3D domain decomposition, P^3 processes

Given a 3D domain (N x N x N), compute the start and end indices of the respective sub-domains.

int xStart=_____,
yStart=_____,
zStart=_____;

int xEnd=_____,
yEnd=_____,
zEnd=_____;



Thank you for your attention!